

Содержание

2. Разработка архитектуры системы	2
2.1. Архитектура итоговой системы	2
2.2. Структуры данных и SQL-схема	4
2.3. Алгоритм поиска и доступности	6
2.4. Атомарное создание заказа	7
2.5. Состояния заказа и демооплаты	10
2.6. Аккаунты и восстановление доступа	10
2.7. Администрирование, фотографии и карты	11
2.8. HTML-документ и передача почтовому клиенту	12
2.9. Подход к тестированию	12
2.10. Выводы по главе	13
Список используемой литературы	14

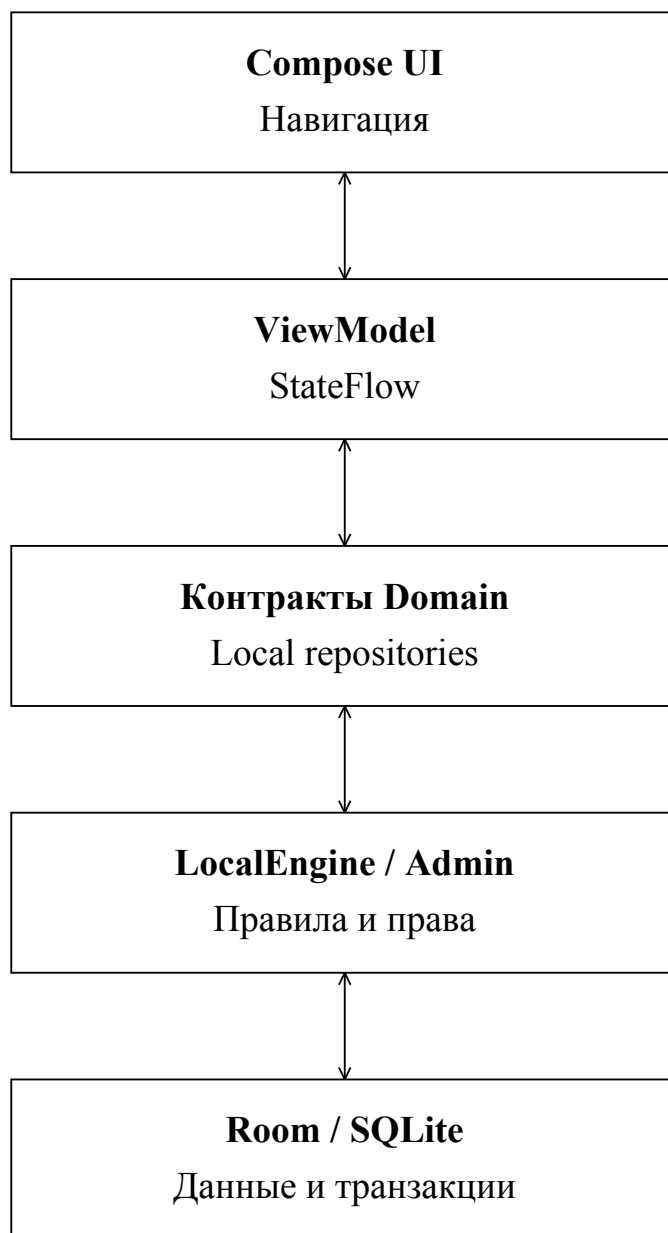
2. Разработка архитектуры системы

2.1. Архитектура итоговой системы

Архитектура Android сохраняет разделение UI-Domain-Data и MVVM. Compose отображает состояние, ViewModel принимает события и публикует StateFlow, репозитории реализуют контракты предметной области. Hilt связывает рабочие контракты с LocalCatalogRepository, LocalAuthRepository и LocalTravelRepository. LocalEngine объединяет предметные правила и транзакции, а LocalDao выполняет SQL. Благодаря этому те же экраны не содержат запросов SQLite и не обращаются к сети напрямую.

Схема, выбранная на этапах 3-6, включала Android, Ktor REST API и PostgreSQL. Её сервер, маршруты, JWT, миграции V1-V5 и PDF-генератор сохраняются в каталоге server. В версии 0.9.0 рабочие Android-репозитории к нему не привязаны. Не следует описывать самостоятельный APK как HTTP-клиент с кэшем: Room здесь основной источник истины. Установка на другой телефон создаёт новую базу, а не подключает его к общей гостиничной системе.

Рисунок 2.1 - Компоненты самостоятельного APK



Mapsforge: локальные assets; сеть не используется.

Mapsforge отображает локальные бинарные map-файлы, подготовленные из OSM XML. Приложение не использует слой скачивания тайлов. Файл копируется из assets во внутреннее хранилище, затем читает MapFile. Нативный MapView размещён в Compose через AndroidView, а при выходе освобождает renderer и cache. Каталог и маркеры используют одинаковые идентификаторы.

Фоновые вычисления выполняются через Coroutines на Dispatchers.IO. Загрузка, ошибка, пустой результат и успешное состояние представлены явно. Независимые задачи поиска и карточки отменяются при новом запросе; поздний результат не заменяет более свежий. Операции заказа не выполняются одновременно одним ViewModel. При смене пользователя очищаются его экранные заказы и документ, но не SQL-история.

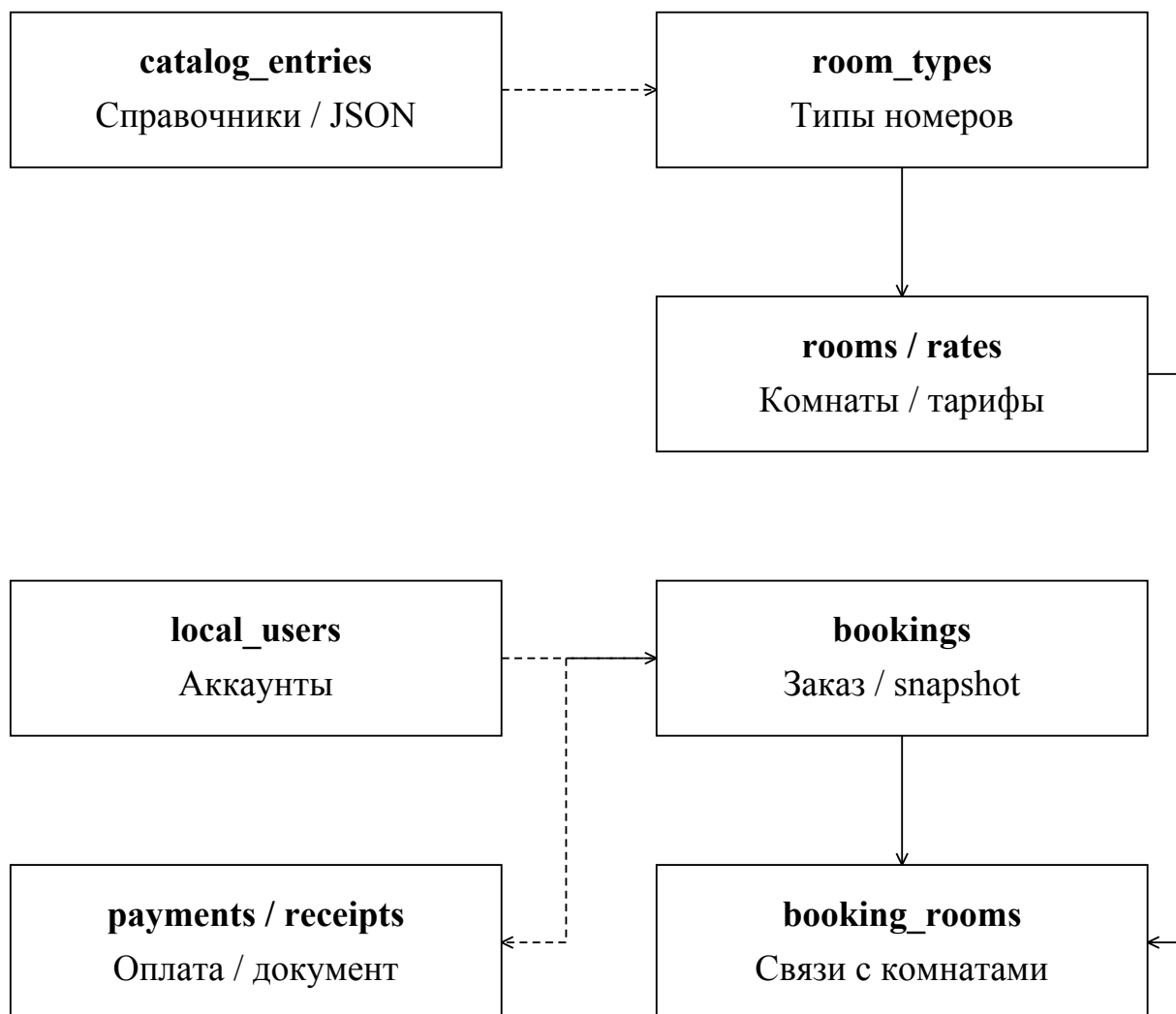
2.2. Структуры данных и SQL-схема

Основная база hotel_offline_09.db использует Room version 1 с экспортом схемы в Git. catalog_entries хранит отдельные записи стран, городов, гостиниц, мест и удобств: id, kind, title, payload, active, version. payload содержит JSON одного объекта, а не общий файл всего каталога. По kind создан индекс. Родительские связи внутри справочных JSON-объектов проверяет LocalAdmin; это не физические внешние ключи отдельных country/city таблиц. Такое проектное упрощение явно отличается от нормализованной серверной PostgreSQL-схемы.

Часто изменяемые остатки, интервалы, пользователи и заказы выделены в типизированные SQL-таблицы. room_types связывает тип с гостиницей и хранит вместимость, площадь, коды удобств и active. rooms содержит тип и номер комнаты; rates - тип, цену, валюту, границы и условия оплаты/отмены. Между rooms/rates и room_types, room_blocks и rooms, booking_rooms и bookings/rooms заданы внешние ключи с RESTRICT. Цены записываются INTEGER, а не REAL.

bookings хранит владельца, гостиницу, тип, даты, idempotency key, fingerprint и JSON-снимок заказа. booking_rooms задаёт выделение физических комнат. Сведения о выбранных услугах входят в неизменяемый BookingSnapshot; отдельной таблицы booking_services в локальной версии нет. Это не следует смешивать с сервером, где такая таблица существует. payments имеет уникальную связь с заказом; receipts хранит исходный оплаченный снимок для повторного HTML-формирования.

Рисунок 2.2 - Основные группы SQL-сущностей



Также: room_blocks, hotel_services, hotel_places, audit.

Пунктир: логическая связь; сплошная: SQL-связь.

Таблица SQLite	Назначение и ограничения
catalog_entries	Справочные записи; PK id, индекс kind, JSON одного объекта, active/version
local_users	Email UNIQUE, роль, Argon2id-хеши пароля/кода, версия сеанса

Таблица SQLite	Назначение и ограничения
room_types / rooms	Типы и физические ресурсы; номер уникален в гостинице по бизнес-правилу
rates / room_blocks	Цены и интервалы; пересечение тарифов/занятых блокировок отклоняется сервисом
hotel_services / hotel_places	Услуги гостиницы и связь с общими местами города
bookings / booking_rooms	Заказ, снимок и атомарное выделение комнат; UNIQUE ownerId/key
payments / receipts	Одна демооплата и один оплаченный снимок на заказ
audit	Локальный журнал действий ADMIN без паролей и восстановительных кодов

Начальный seed выполняется только для новой базы внутри транзакции. Детерминированные идентификаторы обеспечивают воспроизводимость. Маркер __seed09 не позволяет повторному запуску заменить отредактированные сведения или сбросить пароли. При обновлении APK база сохраняется; destructive migration не используется. Будущая смена структуры должна сопровождаться явной Room Migration и тестом перехода. Удаление данных пользователем средствами Android остаётся внешней операцией и уничтожает локальную демонстрацию.

2.3. Алгоритм поиска и доступности

Сначала проверяются существование направления, даты ISO, число гостей и комнат, допустимость фильтров и длина проживания. Начало не может быть в прошлом, конец должен быть позже начала, длительность - от одной до девяноста ночей. На каждую комнату нужен хотя бы один взрослый; необходимая вместимость определяется распределением суммарного числа гостей по одинаковым комнатам.

После фильтрации гостиниц выбираются активные типы, удовлетворяющие вместимости, категории и удобствам. Для типа нужен активный тариф в валюте страны, покрывающий весь интервал. Отдельные тарифы для разных частей поездки в этой версии не объединяются. Активные тарифы одного типа не могут пересекаться: правило проверяется при административной записи внутри общей транзакции.

Комната считается занятой при пересечении интервалов $start < checkOut$ и $end > checkIn$. Из доступного множества исключаются комнаты активных закрытых периодов, подтверждённых заказов и неистёкших `PENDING_PAYMENT`. Отменённый и истёкший резерв ресурс не занимает. Если свободных комнат меньше запрошенного числа, предложение не возвращается. Один физический ресурс должен быть свободен весь период; чередование разных комнат по ночам не используется.

Предложения сортируются устойчиво: при равной цене, рейтинге или расстоянии применяется идентификатор гостиницы. Пагинация берётся после общей сортировки, чтобы повторные страницы не меняли порядок. В локальной реализации DAO читает типизированные SQL-записи, а небольшой начальный каталог фильтруется бизнес-сервисом; сложность пропорциональна числу типов и интервалов. Для коммерческого объёма потребуется перенести фильтрацию в агрегирующие SQL-запросы и общий backend.

2.4. Атомарное создание заказа

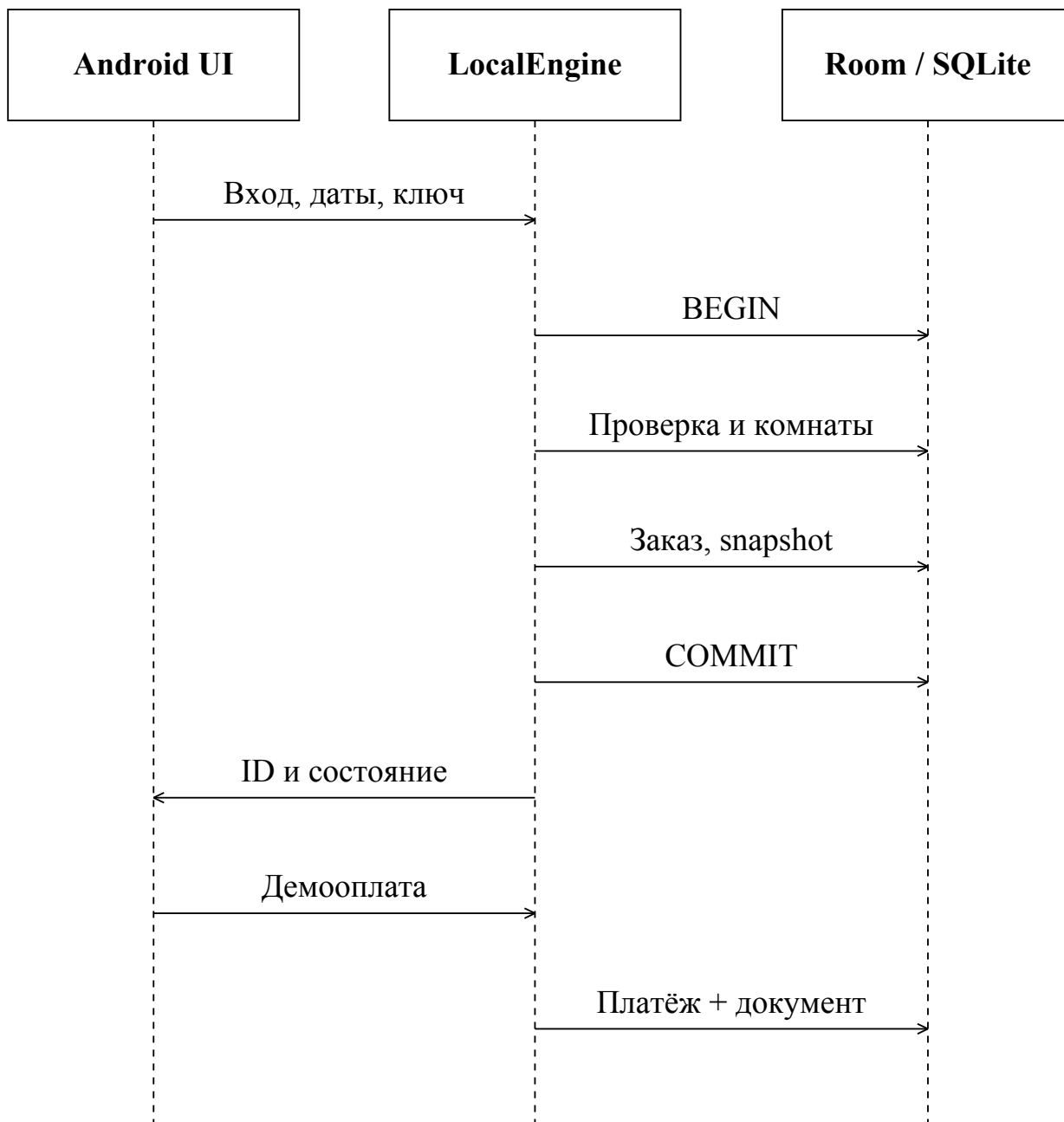
После входа проверяется, что указанный пользователь совпадает с текущим сеансом. Фиксируется `idempotency key` и SHA-256 отпечаток канонического запроса: множества услуг/удобств/звёзд сортируются, сам ключ не входит в отпечаток. Повтор одинакового запроса возвращает прежний заказ. Повтор ключа с другим содержимым отклоняется, поэтому двойное нажатие не создаёт второй заказ.

В одной `Room withTransaction` повторно проверяются действующий каталог, тариф, способ оплаты и количество комнат. Выделяются конкретные свободные ресурсы, рассчитываются проживание и услуги. Проживание равно ночной цене, умноженной на число ночей и комнат. Сумма услуг `PER_NIGHT_ROOM` также умножается на обе величины; `ONCE` добавляется

один раз. Операции используют проверяемые целочисленные умножения и сложения. Если сумма не совпадает с принятой клиентом, возвращается PRICE_CHANGED без создания заказа.

Все записи заказа и выделенных комнат фиксируются вместе. При недостаточном наличии возвращается AVAILABILITY_CHANGED, при исключении транзакция не оставляет частичного заказа. SQLite сериализует запись; параллельные операции не могут обе принять один устаревший остаток. Это механизм локальной базы, а не PostgreSQL row-level locks. В серверной реализации сохранён другой алгоритм блокировки физических комнат по UUID.

Рисунок 2.3 - Последовательность заказа и демооплаты



HTML читается из снимка после успешной транзакции.

Снимок фиксирует название и адрес гостиницы, клиента, комнатную категорию, номера, ночную цену, общую цену проживания, услуги с реквизитами трансфера и часовой пояс. Срок отмены вычисляется при создании и также сохраняется. Последующее редактирование гостиницы, профиля или тарифа не пересчитывает существующий заказ.

2.5. Состояния заказа и демооплаты

ONLINE_DEMO создаёт PENDING_PAYMENT/UNPAID с expiresAt через пятнадцать минут. PAY_AT_HOTEL создаёт CONFIRMED/UNPAID без резерва оплаты. При демооплате повторно проверяются владелец, статус, способ и время; payment, receipt и CONFIRMED/PAID сохраняются одной транзакцией. Одна успешная демооплата на заказ обеспечивается уникальным ограничением. Повтор успешной операции возвращает прежний transactionId и не продлевает резерв.

При достижении expiresAt резерв получает CANCELLED с причиной EXPIRED. Проверка остатков исключает истёкший резерв даже до обновления его текстового статуса. Во время чтения/записи заказов housekeeping обновляет истёкшие резервы и завершившиеся проживания. В автономной версии не обещается ежеминутная работа закрытого процесса Android: после открытия выполняется сверка по текущему времени устройства.

Заезд установлен на 14:00, выезд - 12:00 по IANA-зоне города. Подтверждённый заказ разрешено отменять до cancelUntil; pending-заказ отменяется до оплаты. Оплаченная демонстрационная операция становится REVERSED, но банковский возврат не проводится. После окончания проживания состояние меняется на COMPLETED. Изменение часов телефона может повлиять на эти сроки, что открыто описано в руководстве.

2.6. Аккаунты и восстановление доступа

Email нормализуется и уникален в локальной базе. Пароль хранится как индивидуально посоленный Argon2id-хеш: 19 МиБ памяти, два прохода, параллелизм один [5]. Хеширование выполняется вне UI-потока. Открытый пароль не пишется в базу, лог, SavedState или Git. Внутренний сеанс содержит пользователя и версию; он шифруется AES-GCM неэкспортируемым ключом Android Keystore [6]. Новое пространство noBackup-файлов не удаляет сохранённый серверный сеанс старой версии.

При первом входе создаётся случайный индивидуальный код восстановления. В Room сохраняется его Argon2id-хеш. До подтверждения получения зашифрованная копия хранится в noBackup, чтобы закрытие процесса не лишило пользователя кода. После подтверждения копия

удаляется. Восстановление требует email и код; успешное использование меняет пароль, повышает версию сеанса и выдаёт новый код. Смена пароля требует текущий пароль и тоже заменяет код.

Начальные учебные пароли известны для проверки проекта и не считаются секретами коммерческого сервиса. Они не дают доступа к чужому устройству: каждая база независима. Отсутствие кода и пароля не компенсируется скрытым универсальным входом. Взломанный/rooted телефон и подмена собственной SQLite-базы вне приложения находятся за границей модели угроз учебной демонстрации.

2.7. Администрирование, фотографии и карты

LocalAdmin проверяет роль перед каждой операцией. Формы используют метаданные AdminSchema, а связи выбираются из существующих активных объектов. Валюта тарифа должна совпадать со страной гостиницы; вместимость и координаты ограничены; пересекающиеся тарифы и закрытие занятой комнаты отклоняются. Версия записи предотвращает незаметное сохранение формы поверх более свежего изменения. Архивирование сохраняет идентификаторы и историю.

Фотография выбирается системным Photo Picker, копируется во внутреннее хранилище, проверяется по объёму и числу пикселей, уменьшается и перекодируется в JPEG без метаданных EXIF. SQL хранит только относительное случайное имя. Произвольный путь пользователя не принимается. Импортированные фото отображаются перед встроенными иллюстрациями; перестановка первого фото не требует повторного импорта. При неудачной записи новая копия удаляется, старые изображения не затираются.

Шесть map-файлов подготовлены Mapsforge map-writer из ограниченных OSM API /map выгрузок. Скрипт подготовки выполняется разработчиком, не телефоном; общедоступные tile-server не скачиваются массово. manifest.json содержит границы, URL, размер и SHA-256 каждого файла. На карте сохраняется атрибуция OpenStreetMap contributors и ODbL [7]. LGPL-лицензия библиотеки и ODbL карт относятся к соответствующим компонентам, а не объявляют весь каталог реальными OSM-объектами.

Отдельная запись администратора может находиться за пределами встроенного покрытия. Её список, описание и координаты доступны, но глобальная карта не подгружается скрыто. Расстояние вычисляется формулой гаверсинуса и означает прямую географическую дистанцию; маршрутизация и GPS-навигация не реализованы.

2.8. HTML-документ и передача почтовому клиенту

Документ формируется после успешной транзакции из оплаченного снимка. Его повторное открытие не создаёт платёж и не пересчитывает цену. В HTML включаются заказ, клиент, гостиница, номера, даты, услуги, сумма, валюта, способ и `demo transactionId`. Пользовательский текст экранируется. `WebView` отключает `JavaScript`, сеть и доступ к файловым `URI`; для отображения не загружаются внешние шрифты или изображения.

Системный `SAF` позволяет сохранить HTML без разрешения на всё хранилище. `FileProvider` выдаёт почтовому приложению временное `read`-разрешение на выбранное вложение. В `Gmail/Mail.ru` передаются адрес, тема, пояснение и HTML-файл; окончательную отправку выполняет пользователь. Наличие интернета не означает наличие настроенной почты, а открытие `chooser` не означает доставку. При отсутствии почтового клиента показывается объяснение и возможность сохранить файл. Автоматический `SMTP` и восстановление через `email` не входят в самостоятельную версию.

2.9. Подход к тестированию

Модульные тесты проверяют доменные ограничения, `ViewModel`, отмену устаревших запросов и защиту повторных действий. `Room instrumentation` тестирует настоящий `Android SQLite` в отдельной `in-memory` базе, не очищая производственные данные. `Fake Clock` позволяет проверить точную границу пятнадцати минут без ожидания. Параллельные корутины проверяют атомарное выделение нескольких комнат.

Интерфейсный сценарий через настоящую `MainActivity`, `Hilt`, `Room` и `Keystore` охватывает вход клиента, поиск, обе оплаты, HTML, карту, пересоздание `Activity` и административный экран. Отдельные `Compose`-тесты проверяют формы, фильтры, навигацию и состояния. Испытания запускаются

только на явно заданном emulator-serial; общий connectedDebugAndroidTest не используется, чтобы не затронуть личный телефон.

Три конфигурации функциональных испытаний, результаты запусков и ограничения оборудования фиксируются в docs/STAGES_7_9_VERIFICATION.md. Сборка, lint, Dokka, подпись и контрольная сумма APK проверяются отдельно: успешный unit-тест не доказывает работоспособность установки, а Windows CI не заменяет испытания на Android. Личные данные и секреты не публикуются в протоколе.

2.10. Выводы по главе

Разработанная архитектура отделяет Compose-интерфейс и состояние экранов от предметных правил и SQL-хранилища. Room обеспечивает атомарность заказа и оплаты, неизменяемые снимки сохраняют историю после редактирования каталога, а повторная проверка доступности предотвращает выделение одной комнаты пересекающимся заказам. Mapsforge и поставляемые карты позволяют исследовать центральные районы шести городов без сетевого сервиса.

Самостоятельная версия является локальной демонстрационной системой, а не распределённым гостиничным агрегатором. Ограничения часов устройства, размеров карты, ручной отправки писем и независимости баз явно отражены в интерфейсе и документации. Сохранённый серверный вариант не подменяет эти условия: для общего online-режима потребуется отдельная архитектура синхронизации и эксплуатации. Выбранное разделение компонентов обеспечивает основу для дальнейшего расширения и проверяемых сценариев пользователя и администратора.

Список используемой литературы

1. Booking.com. Mobile application.
URL: <https://www.booking.com/apps.html>
(дата обращения: 05.10.2026).
2. Островок. Бронирование гостиниц и способы оплаты.
URL: <https://ostrovok.ru/>
(дата обращения: 05.10.2026).
3. Tripadvisor. Hotels, restaurants and attractions.
URL: <https://www.tripadvisor.com/>
(дата обращения: 05.10.2026).
4. Google Maps Help. Find nearby places.
URL: <https://support.google.com/maps/answer/144349>
(дата обращения: 05.10.2026).
5. OWASP. Password Storage Cheat Sheet.
URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
(дата обращения: 05.10.2026).
6. Android Developers. Android Keystore.
URL: <https://developer.android.com/privacy-and-security/keystore>
(дата обращения: 05.10.2026).
7. OpenStreetMap. Copyright and License.
URL: <https://www.openstreetmap.org/copyright>
(дата обращения: 05.10.2026).
8. Android Developers. Room and asynchronous queries.
URL: <https://developer.android.com/training/data-storage/room/async-queries>
(дата обращения: 05.10.2026).
9. Mapsforge. Integration guide.
URL: <https://github.com/mapsforge/mapsforge/blob/master/docs/Integration.md>
(дата обращения: 05.10.2026).

10. Kotlin. Documentation tool Dokka.

URL: <https://kotlinlang.org/docs/dokka-introduction.html>

(дата обращения: 05.10.2026).

11. Семакова А. Введение в разработку приложений для смартфонов на ОС Android. М.: ИНТУИТ, 2016.

12. Гриффитс Д., Гриффитс Д. Head First. Программирование для Android. 2-е изд. СПб.: Питер, 2018. 912 с.